

ANÁLISIS DEL RETO 1

Juan Sebastián Suárez Santos, 202610848, j.suarezs2@uniandes.edu.co

Samuel Rodríguez Romero, 202610831, s.rodriguez23456712@uniandes.edu.co

Nicolas Gómez Ayala, 202024568, nm.gomeza1@Uniandes.edu.co

Requerimiento 1

Descripción

Entrada	Parametros necesarios para resolver el requerimiento
Salidas	Respuesta esperada del algoritmo
Implementado (Sí/No)	Si se implemento y quien lo hizo

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Paso 1	$O(\dots)$
Paso 2	$O(\dots)$
Paso 3	$O(\dots)$
Paso...	$O(\dots)$
TOTAL	$O(\dots)$

Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el analisis de complejidad.

Requerimiento Ejemplo

Descripción

```
def get_data(data_structs, id):  
    """  
    Retorna un dato a partir de su ID  
    """  
    pos_data = lt.isPresent(data_structs["data"], id)  
    if pos_data > 0:  
        data = lt.getElement(data_structs["data"], pos_data)  
        return data  
    return None
```

Este requerimiento se encarga de retornar un dato de una lista dado su ID. Lo primero que hace es verificar si el elemento existe. Dado el caso que exista, retorna su posición, lo busca en la lista y lo retorna. De lo contrario, retorna None.

Entrada	Estructuras de datos del modelo, ID.
Salidas	El elemento con el ID dado, si no existe se retorna None
Implementado (Sí/No)	Si. Implementado por Juan Andrés Ariza

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Buscar si el elemento existe (isPresent)	$O(n)$
Obtener el elemento (getElement)	$O(1)$
TOTAL	$O(n)$

Análisis

A pesar de que obtener un elemento en un *ArrayList*, dada su posición, tiene complejidad constante, la implementación de este requerimiento tiene un orden lineal $O(n)$. Esto debido a que, lo primero que se hace es verificar si el elemento hace parte de la lista. Específicamente, a la hora de buscar un elemento en una lista, en el peor de los casos es necesario recorrer toda la lista, es decir, complejidad lineal.

REQUERIMIENTO 1

Descripción: Se recorre una sola vez el catálogo almacenado como array. Por cada pedido cuyo campo Product coincide con el nombre buscado, se acumulan sumas y se actualizan mínimos/máximos de Price_per_Box, Discount_Pct, Boxes_Shipped y Marketing_Spend. También se lleva un diccionario de conteo por año (a partir de los primeros 4 caracteres de Order_Date) y se guarda el índice del pedido con mayor y menor Amount visto hasta el momento. Al terminar el recorrido, se calcula el año más frecuente y se recuperan los pedidos ganadores por índice.

Entrada: Catalog

Salidas: Tiempo de ejecución, total de pedidos del producto, promedio, mínimo, máximo de Price_per_Box, Discount_Pct, Boxes_Shipped y Marketing_Spend, año con más pedidos, información del pedido de mayor Amount y de menor Amount

Implementado: Si, por Samuel Rodríguez

Análisis de complejidad

Obtener el tamaño del catálogo	$O(1)$
Recorrer todo el catálogo	$O(n)$
Filtrar y acumular sumas/mín/máx por pedido que cumple	$O(1)$ por iteración
Actualizar el diccionario de conteo por año	$O(1)$ amortizado por iteración
Recorrer el diccionario de años para hallar el más frecuente	$O(k)$, $k \leq n$
Obtener los pedidos de mayor/menor Amount por índice	$O(1)$ cada uno
TOTAL	$O(n)$

Análisis

la complejidad del requerimiento queda dominada por el único recorrido lineal del catálogo. Aunque hay varias operaciones distintas dentro del ciclo, ninguna está anidada dentro de otro recorrido del catálogo, así que cada una aporta a lo sumo $O(1)$ por iteración y el total permanece en $O(n)$.

REQUERIMIENTO 4

Descripción: Se recorre la lista enlazada siguiendo directamente los punteros next desde catalog["first"], sin usar sl.get_element(catalog, i) dentro de un ciclo. Por cada nodo cuyo pedido cumple Product == producto y Country == país, se acumulan las sumas de Price_per_Box, Discount_Pct, Marketing_Spend y

Boxes_Shipped y se mantienen los dos pedidos de mayor Amount vistos hasta el momento. Al terminar el recorrido se calculan los promedios dividiendo entre el total de pedidos filtrados.

Entrada: Catálogo

Salidas: Tiempo de ejecución, total de pedidos que cumplen el filtro, promedio de Price_per_Box, Discount_Pct, Marketing_Spend y Boxes_Shipped, y los 2 pedidos de mayor Amount.

Implementado: Si

Análisis de complejidad

Acceder al primer nodo de la lista enlazada	$O(1)$
Recorrer la lista completa siguiendo punteros next	$O(n)$
Filtrar y acumular sumas por nodo que cumple	$O(1)$ por iteración
Comparar y actualizar top_amount_1 / top_amount_2	$O(1)$ por iteración
Calcular los 4 promedios finales	$O(1)$
TOTAL	$O(n)$

Análisis

A diferencia de recorrer con `sl.get_element(catalog, i)` dentro de un `for`, aquí se avanza directamente con `nodo_actual = nodo_actual["next"]`, lo que garantiza un único recorrido lineal real. Mantener los dos mejores candidatos por Amount evita cualquier operación de ordenamiento. La complejidad total queda en $O(n)$, igual que `req_1`.